

Predicting Air Pressure System Failures in Heavy Trucks

A Comparative Machine Learning Study · Rymma Won· CSCI 460 — Machine Learning · Spring 2026 · Scania APS Dataset (IDA 2016)

INTRODUCTION & OBJECTIVE

Heavy trucks rely on the Air Pressure System (APS) to power brakes and gear changes. APS failures cause brake loss — a critical safety hazard — or costly roadside breakdowns.

Research Question:

Can sensor data from Scania trucks predict whether a failure came from the APS — and which ML model minimizes the real-world cost of misclassification?

- Three reasons this is hard:
- Severe class imbalance — only 1.7% of trips are APS failures (59:1 ratio)
 - 170 anonymized sensor features, many correlated or partially missing
 - A missed failure costs 50x more than a false alarm

Official Cost Formula:
Total Cost = 10 × False Positives + 500 × False Negatives

Missing one real failure costs as much as 50 false alarms. Recall is the dominant metric.

DATASET OVERVIEW

Split	Total Rows	APS Failures	Rate
Training	60,000	1,000	1.7%
Test	16,000	375	2.3%

- 170 anonymized sensor features — histogram bins, counters, pressure thresholds
- ~14% of values missing, encoded as the string 'na'
- 3 features had >80% missing values — dropped before modeling
- APS-failure trucks show heavier missingness in specific sensors

EXPLORATORY DATA ANALYSIS

Finding	Action Taken
59:1 class imbalance	Class reweighting in all models
~850,000 missing values	Median imputation — training set only
Right-skewed distributions	StandardScaler on all features
High correlation (r > 0.90)	PCA for Logistic Reg., KNN, SVM
Outliers are real events	Retained — not removed

A model predicting 'no failure' for everything is 98.3% accurate but catches zero real failures — accuracy is useless here.

METHODOLOGY

- Preprocessing Pipeline:
- Encode Target**
'pos' → 1 (APS failure), 'neg' → 0
 - Drop Near-Empty Features**
Remove 3 features with >80% missing values
 - Median Imputation**
Fill NAs with training-set median — no data leakage
 - StandardScaler**
Zero-mean, unit-variance — essential for LR, KNN, SVM
 - PCA (LR, KNN, SVM only)**
Keep 95% variance (~100 components); removes multicollinearity
 - Class Reweighting**
class_weight='balanced' or scale_pos_weight=59 for XGBoost

- Six Models Trained & Compared:
- Logistic Regression:**
Linear on PCA features; class_weight='balanced'
- K-Nearest Neighbors:**
PCA features; k and distance metric tuned
- Decision Tree:**
Full 167 features; depth, leaf size, criterion tuned
- Random Forest:**
Full features; 300 trees; ensemble bagging
- XGBoost:**
Full features; scale_pos_weight=59; boosting
- Linear SVM:**
PCA features; C regularization tuned

Tuning: Randomized Search · 3-fold stratified CV · scored on F1 · test set held out until final evaluation.

RESULTS

All six models — test set — ranked by Domain Cost (lowest = best):

Model	Recall	F1	ROC-AUC	Precision	Cost
★ Logistic Regression	0.893	0.609	0.962	0.462	23,900
Decision Tree	0.813	0.613	0.900	0.492	38,150
XGBoost	0.789	0.822	0.995	0.858	39,990
Linear SVM	0.787	0.699	0.916	0.629	41,740
Random Forest	0.757	0.763	0.993	0.770	46,350
K-Nearest Neighbors	0.560	0.664	0.858	0.814	82,980

★ Gold = best domain cost Red = worst (KNN) XGBoost = best F1 & AUC

KEY FINDINGS & CONCLUSIONS

The simplest model won on cost: Logistic Regression achieved a domain cost of 23,900 — the lowest of all models. It won because it caught 89.3% of real failures (highest recall). Given the 50:1 penalty, that recall advantage outweighed all other metrics.

Why XGBoost lost despite better F1 and AUC: XGBoost flags a truck only when confidence exceeds 50% by default. That conservatism missed ~79 failures vs. LR's ~40. At 500 units per miss, 39 extra misses = ~19,500 extra units, erasing its false-alarm savings. Lowering its threshold to ~25% would likely beat LR.

- Conclusions:
- 87% cost reduction — best model (23,900) vs. flagging nothing (187,500)**
 - Cost structure drives model choice — recall matters more than F1 or accuracy here
 - Preprocessing was essential — reweighting, imputation, scaling, and PCA all contributed
 - KNN failed badly — recall 0.560; instance-based methods cannot handle 59:1 imbalance
 - ~20–30 histogram sensors carried most of the predictive signal in RF and XGBoost

- Future Work:
- Tune decision thresholds — XGBoost/RF have AUC > 0.99; lower flag threshold boosts recall without retraining
 - SMOTE oversampling — generate synthetic APS failure examples to help models learn the rare class
 - Missingness as a feature — binary 'was this sensor missing?' columns; silence may carry signal
 - Time-series modeling — incorporate historical sensor trends to detect APS degradation earlier